GLOBAL VARIABLE

blockhash(uint blockNumber) returns (bytes32): hash of the given block when blocknumber is one of the 256 most recent blocks; otherwise returns zero
block.basefee (uint): current block's base fee (EIP-3198 and EIP-1559)
block.chainid (uint): current chain id
block.coinbase (address payable): current block miner's address
block.difficulty (uint): current block difficulty
block.gaslimit (uint): current block gaslimit
block.number (uint): current block number
block.timestamp (uint): current block timestamp as seconds since unix epoch
gasleft() returns (uint256): remaining gas
msg.data (bytes calldata): complete calldata
msg.sender (address): sender of the message (current call)
msg.sig (bytes4): first four bytes of the calldata (i.e. function identifier)
msg.value (uint): number of wei sent with the message
tx.gasprice (uint): gas price of the transaction
tx.origin (address): sender of the transaction (full call chain)

Code Eater

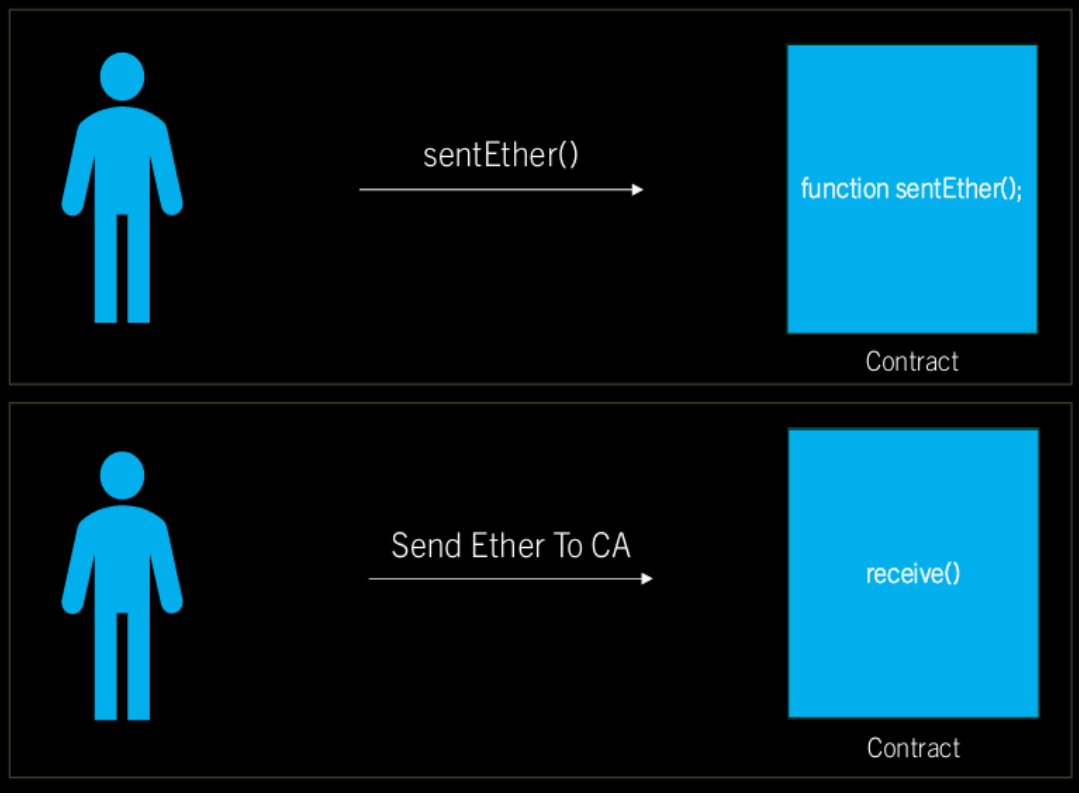
* fallback function → kuch gadbad hai to execute ho & agar ether transection hai to waste na jaye or contract ke account mai aa jaye.

* receive function → contract mai agar payable function nahi hai to yeh waste nahi jane deta ether.

FALLBACK

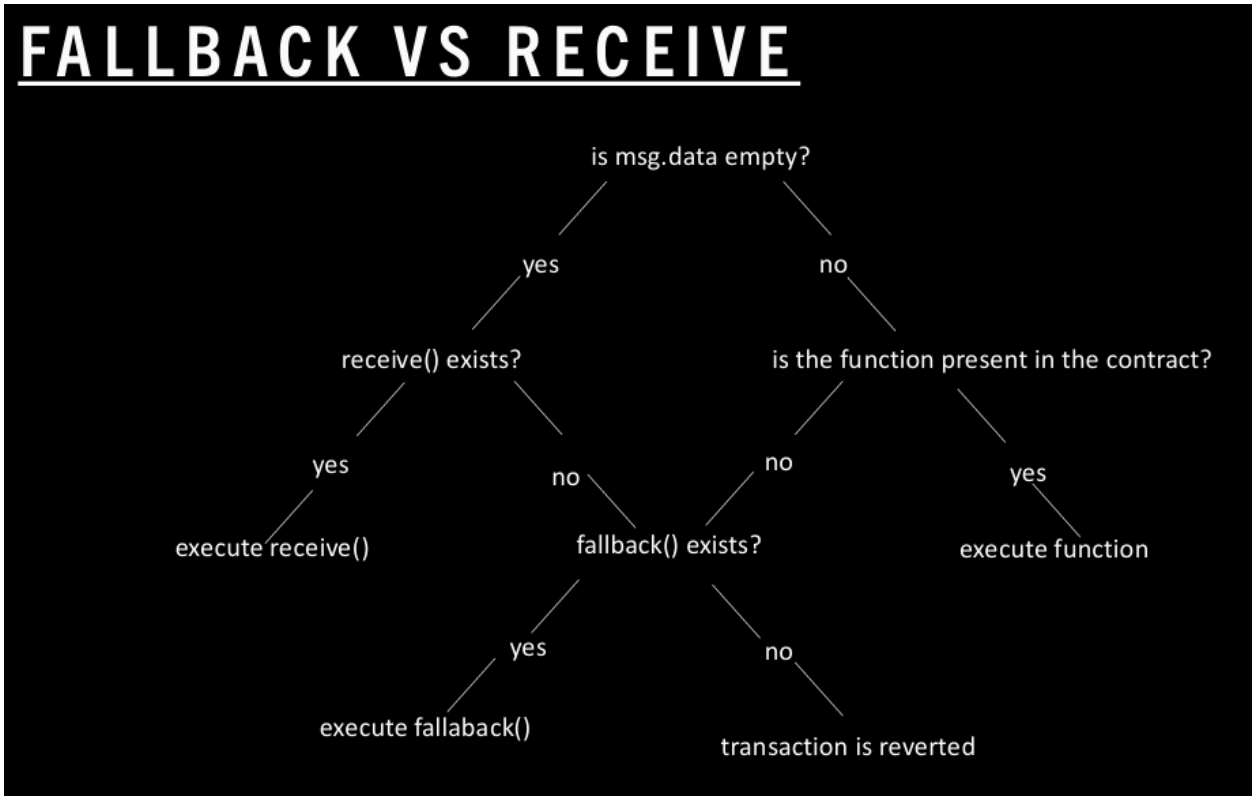
- The fallback function is executed on a call to the contract if none of the other functions match the given function signature, or if no data was supplied at all and there is no receive Ether function.
- Fallback function is optionally payable and must have external visibility.
- A contract can have at most one fallback function, declared using either fallback () external [payable] or fallback (bytes calldata input) external [payable]returns (bytes memory output)
- A fallback function can be virtual, can override and can have modifiers.

WAYS TO RECEIVE ETHER



RECEIVE

- A contract can have at most one receive function, declared using receive() external payable { ... }.
- This function cannot have arguments, cannot return anything, and must have external visibility and payable state mutability.
- It can be virtual, can override and can have modifiers.
- If neither a receive Ether nor a payable fallback function is present, the contract cannot receive Ether through regular transactions and throws an exception.
- Using payable fallback functions for receiving Ether is not recommended.



WALLET SIR CODE

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract SimpleWallet {

    address public owner;
    string public str;

    event Transfer(address receiver,uint amount);
    event Receive(address sender,uint amonut);

    constructor(){
        owner=msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner,"You don't have access");
        _;
    }
```

```
    /**Contract related functions**/
    function transferToContract() external payable  {
       str="Amount transferred to the contract";
    }


    function transferToUserViaContract(address payable _to, uint
_weiAmount) external onlyOwner {
        require(address(this).balance>=_weiAmount,"Insufficient Balance");
        _to.transfer(_weiAmount);
        emit Transfer(_to,_weiAmount);
    }


    function withdrawFromContract(uint _weiAmount) external onlyOwner {
       require(address(this).balance >= _weiAmount, "Insuffficient
balance");
       payable(owner).transfer(_weiAmount);
    }


    function getContractBalanceInWei() external view returns (uint) {
         return address(this).balance;
    }

     /**User related functions**/
    function transferToUserViaMsgValue(address _to) external payable {
       require(address(this).balance>=msg.value,"Insufficient Balance");
       payable (_to).transfer(msg.value);
    }


    function receiveFromUser() external payable {
      require(msg.value>0,"Wei Value must be greater than zero");
      payable(owner).transfer(msg.value);
    }

    function getOwnerBalanceInWei() external view returns(uint){
       return owner.balance;
```

```
    }

    receive() external payable {
       emit Receive(msg.sender, msg.value);
    }

    fallback() external {

    }


}

//Add the following features
//1.List of receivers with amount
//2.List of transfers with amount
```

WALLET IMPROVEMENT CODE

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

//Transaction - from,to,timings,amount
contract SimpleWallet {
    struct Transaction {
        address from;
        address to;
        uint256 timestamp;
        uint256 amount;
    }

    Transaction[] public transactionHistory;
```

```
    address public owner;
    string public str;
    bool public stop;
    event Transfer(address receiver, uint256 amount);
    event Receive(address sender, uint256 amonut);
    event ReceiveUser(address sender, address receiver, uint256 amount);

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "You don't have access");
        _;
    }
    mapping(address => uint256) suspiciousUser;

    modifier getSuspiciousUser(address _sender) {
        require(
            suspiciousUser[_sender] < 5,
            "Activity found suspicious, Try later"
        );
        _;
    }

    modifier isEmergencyDeclared() {
        require(stop == false, "Emergency declared");
        _;
    }

    function toggleStop() external onlyOwner {
        stop = !stop;
    }

    function changOwner(address newOwner) public onlyOwner
isEmergencyDeclared {
        owner = newOwner;
    }

    /**Contract related functions**/
```

```
    function transferToContract(uint256 _startTime)
        external
        payable
        getSuspiciousUser(msg.sender)
    {
        require(block.timestamp > _startTime, "send after start time");
        transactionHistory.push(
            Transaction({
                from: msg.sender,
                to: address(this),
                timestamp: block.timestamp,
                amount: msg.value
            })
        );
    }

    function transferToUserViaContract(address payable _to, uint256
_weiAmount)
        external
        onlyOwner
    {
        require(address(this).balance >= _weiAmount, "Insufficient
Balance");
        require(_to != address(0), "Adress format incorrect");
        _to.transfer(_weiAmount);
        transactionHistory.push(
            Transaction({
                from: msg.sender,
                to: _to,
                timestamp: block.timestamp,
                amount: _weiAmount
            })
        );
        emit Transfer(_to, _weiAmount);
    }

    function withdrawFromContract(uint256 _weiAmount) external onlyOwner {
        require(address(this).balance >= _weiAmount, "Insuffficient
balance");
        payable(owner).transfer(_weiAmount);
```

```
        transactionHistory.push(
            Transaction({
                from: address(this),
                to: owner,
                timestamp: block.timestamp,
                amount: _weiAmount
            })
        );
    }

    function getContractBalanceInWei() external view returns (uint256) {
        return address(this).balance;
    }

    /**User related functions**/
    function transferToUserViaMsgValue(address _to) external payable {
        require(address(this).balance >= msg.value, "Insufficient
Balance");
        require(_to != address(0), "Adress format incorrect");
        payable(_to).transfer(msg.value);
        transactionHistory.push(
            Transaction({
                from: msg.sender,
                to: _to,
                timestamp: block.timestamp,
                amount: msg.value
            })
        );
    }

    //event - sender,receiver, amount
    function receiveFromUser() external payable {
        require(msg.value > 0, "Wei Value must be greater than zero");
        payable(owner).transfer(msg.value);
        emit ReceiveUser(msg.sender, owner, msg.value);
        transactionHistory.push(
            Transaction({
                from: msg.sender,
                to: owner,
                timestamp: block.timestamp,
```

```
                amount: msg.value
            })
        );
    }

    function getOwnerBalanceInWei() external view returns (uint256) {
        return owner.balance;
    }

    receive() external payable {
        transactionHistory.push(
            Transaction({
                from: msg.sender,
                to: address(this),
                timestamp: block.timestamp,
                amount: msg.value
            })
        );
        emit Receive(msg.sender, msg.value);
    }

    function suspiciousActivity(address _sender) public {
        suspiciousUser[_sender] += 1;
    }

    fallback() external {
        suspiciousActivity(msg.sender);
    }

    function getTransactionHistory()
        external
        view
        returns (Transaction[] memory)
    {
        return transactionHistory;
    }

    function emergencyWithdrawl() external {
        require(stop == true, "Emergency not declared");
        payable(owner).transfer(address(this).balance);
```

```
    }
}
// n Solidity, the require statement is used for input validation and
conditional checks. If the first condition (boolean expression) evaluates
to false, the transaction is reverted and the provided error message
(string) is returned.

//Add the following features
//1.Setting and Changing Owner done
//2.Emergency Stop
//3. Emergency Withdrawl
//4. Check for invalid address done
//5. Transaction history - from,to,amount,timestamp done
```

POLYMORPHISM

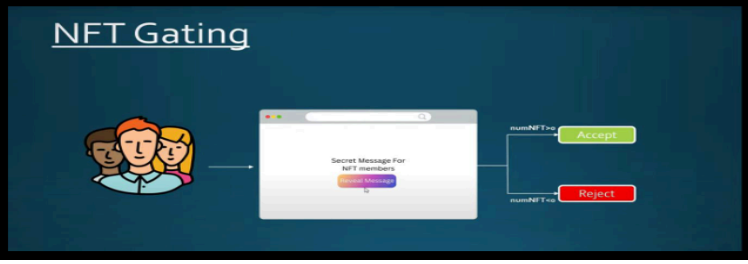
- In Solidity, function polymorphism is achieved through method overloading, where multiple functions can have the same name but differ in the number or types of their parameters.

```
function add(uint a, uint b) public pure returns (uint) {
    return a + b;
}

function add(uint a, uint b, uint c) public pure returns (uint) {
    return a + b + c;
}
```

EVENTS

- Events are defined in Solidity contracts and can be emitted during contract execution. When an event is emitted, it causes the arguments passed to the event to be stored in the transaction's log.
- Logs are a special data structure in the Ethereum blockchain that is associated with the transaction and not stored as part of the contract's state.



EVENT BENEFITS

- **Logging and Monitoring:** Events allow developers to log activities and changes in the contract. This is useful for debugging and monitoring the behavior of smart contracts.
- **Efficient Storage:** Event data is stored in the transaction logs, which are cheaper in terms of gas costs compared to storing data on the blockchain itself.
- **Data Availability:** Events make data available to external systems without requiring them to poll the contract's state continuously.

LIBRARY

- Libraries in solidity are similar to contracts that contain reusable codes.

Importance of Libraries:

- **Code Reusability**: Libraries enable developers to write code once and use it across multiple contracts, reducing redundancy and promoting DRY (Don't Repeat Yourself) principles.
- **Gas Efficiency**: By linking to a deployed library, contracts can call library functions without embedding the library code, which saves gas costs associated with contract deployment and execution.
- **Modularity and Maintainability**: Libraries help keep the codebase organized and modular, making it easier to read, understand, and maintain.
- **Security**: By using well-tested and audited libraries, developers can reduce the risk of introducing bugs and vulnerabilities in their contracts.

LIBRARY

Things to know about of Libraries:

- Functions of the library can be called directly when they do not modify the state variables.
- The library does not have state variables (not entirely true you can read this – article)
- It cannot inherit any element and cannot be inherited.
- Fallback or payable functions also cannot be created inside the library as it cannot store ethers.
- A library is embedded into the contract if all library functions are internal.
- The library must be deployed and then linked before the contract is deployed.

EMBEDDED LIBRARY

Embedded libraries are included directly within the contract's bytecode. When you deploy a contract that uses an embedded library, the library's code becomes part of the contract's bytecode. This means that the library functions are embedded within the contract itself and do not require separate deployment.

A library is embedded into the contract if all library functions are internal.

Advantages of Embedded Libraries

- **Ease of Use**: Since the library code is embedded within the contract, there is no need to manage separate deployments for the library and the contract.
- **Gas Efficiency**: Embedded libraries can save gas costs associated with external calls to library functions, as the function calls are internal.

EMBEDDED LIBRARY

```
library Math {
    function add(uint256 a, uint256 b) internal pure returns (uint256) {    infi
        return a + b;
    }

}

contract Calculator {
    using Math for uint256;

    function calculate(uint256 a, uint256 b) public pure returns (uint256 sum) {
        sum = a.add(b);
    }
}
```

Code - Click Here